



Actividad guiada: Ejercicio autoinstruccional

Imagina que trabajas como analista de datos en una empresa de retail. Te han pedido identificar productos con precios superiores al promedio de su categoría, auditar sueldos que superen el percentil 90 de la empresa, ajustar el stock de productos con ventas superiores al promedio, y depurar pedidos cancelados cuyos clientes ya no existen.

Para resolverlo deberás aplicar subqueries en SQL (SELECT, INSERT, UPDATE, DELETE), demostrando dominio de sentencias anidadas.

Objetivo y preparación de la actividad

Objetivo de la actividad

- Reconocer casos en que las subqueries son útiles.
- Construir subqueries en SELECT, INSERT, UPDATE y DELETE.
- Validar cómo se aplican para obtener y modificar información realista.

Materiales y preparación previa (checklist)

- ✓ Acceso a PostgreSQL o cliente SQL (pgAdmin, DBeaver).
- ✓ Tablas simuladas: productos, empleados, ventas, pedidos, clientes.
- ✓ Conocimiento previo de SELECT, INSERT, UPDATE, DELETE básicos.

Requisitos técnicos y datos

Base de datos con al menos tablas de ejemplo (pueden ser vacías o con registros mínimos). Editor SQL.

Instrucciones paso a paso (30 min total)

01

Explora el caso SELECT (7 min)

Construye una subquery correlacionada:

```
SELECT p.id_producto, p.nombre, p.categoria, p.precio
FROM productos p
WHERE p.precio > (
  SELECT AVG(p2.precio)
  FROM productos p2
  WHERE p2.categoria = p.categoria
);
```

Micro-check: ¿Los productos listados cumplen la condición?

03

Realiza un UPDATE con subquery (8 min)

Reduce stock de productos con ventas superiores al promedio:

```
UPDATE productos
SET stock = stock * 0.9
WHERE id_producto IN (
  SELECT id_producto
  FROM ventas
  GROUP BY id_producto
  HAVING SUM(cantidad) >
  (SELECT AVG(total_ventas)
   FROM (SELECT SUM(cantidad) AS total_ventas
        FROM ventas GROUP BY id_producto) v)
);
```

Micro-check: ¿El stock se redujo solo en productos de alta venta?

02

Aplica un INSERT con subquery escalar (7 min)

Inserta en empleados_auditoria a quienes superen el percentil 90:

```
INSERT INTO empleados_auditoria (id_empleado, nombre,
salario, fecha_auditoria)
SELECT id, nombre, salario, CURRENT_DATE
FROM empleados
WHERE salario > (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY
salario)
FROM empleados
);
```

Micro-check: ¿Se insertaron solo los salarios altos?

04

Ejecuta un DELETE con NOT EXISTS (8 min)

Borra pedidos cancelados sin cliente asociado:

```
DELETE FROM pedidos p
WHERE p.estado = 'Cancelado'
AND NOT EXISTS (
  SELECT 1 FROM clientes c WHERE c.id_cliente = p.id_cliente
);
```

Micro-check: ¿Se eliminaron solo los pedidos huérfanos?

Criterios de éxito y errores comunes

Éxito

- La consulta devuelve o modifica solo lo esperado.
- Cada subquery está correctamente anidada.
- Se usan alias y condiciones de correlación cuando corresponde.
- Evidencia entregable: 4 sentencias SQL con resultados o capturas de ejecución.

Tiempo total: 30'

Errores comunes

- Omitir alias en subqueries derivadas.
- Subquery escalar devolviendo más de 1 fila.
- No verificar condición de correlación (se afectan más filas de las esperadas).

SOLUCIÓN

```
-- 1) SELECT con subquery correlacionada
SELECT p.id_producto, p.nombre, p.categoria, p.precio
FROM productos p
WHERE p.precio > (
  SELECT AVG(p2.precio)
  FROM productos p2
  WHERE p2.categoria = p.categoria
);
```

```
-- 2) INSERT con subquery escalar
INSERT INTO empleados_auditoria (id_empleado, nombre, salario, fecha_auditoria)
SELECT id, nombre, salario, CURRENT_DATE
FROM empleados
WHERE salario > (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salario)
  FROM empleados
);
```

```
-- 3) UPDATE con subquery
UPDATE productos
SET stock = stock * 0.9
WHERE id_producto IN (
  SELECT id_producto
  FROM ventas
  GROUP BY id_producto
  HAVING SUM(cantidad) >
  (SELECT AVG(total_ventas)
   FROM (SELECT SUM(cantidad) AS total_ventas
        FROM ventas GROUP BY id_producto) v)
);
```

```
-- 4) DELETE con NOT EXISTS
DELETE FROM pedidos p
WHERE p.estado = 'Cancelado'
AND NOT EXISTS (
  SELECT 1 FROM clientes c WHERE c.id_cliente = p.id_cliente
);
```